

STOLENT MOUNT - WRITEUP

<https://tryhackme.com/room/hfb1stolenmount>



An intruder has infiltrated our network and targeted the NFS server where the backup files are stored. A classified secret was accessed and stolen. The only trace left behind is a packet capture (PCAP) file recorded during the incident. Your mission, should you accept it, is to discover the contents of the stolen data.

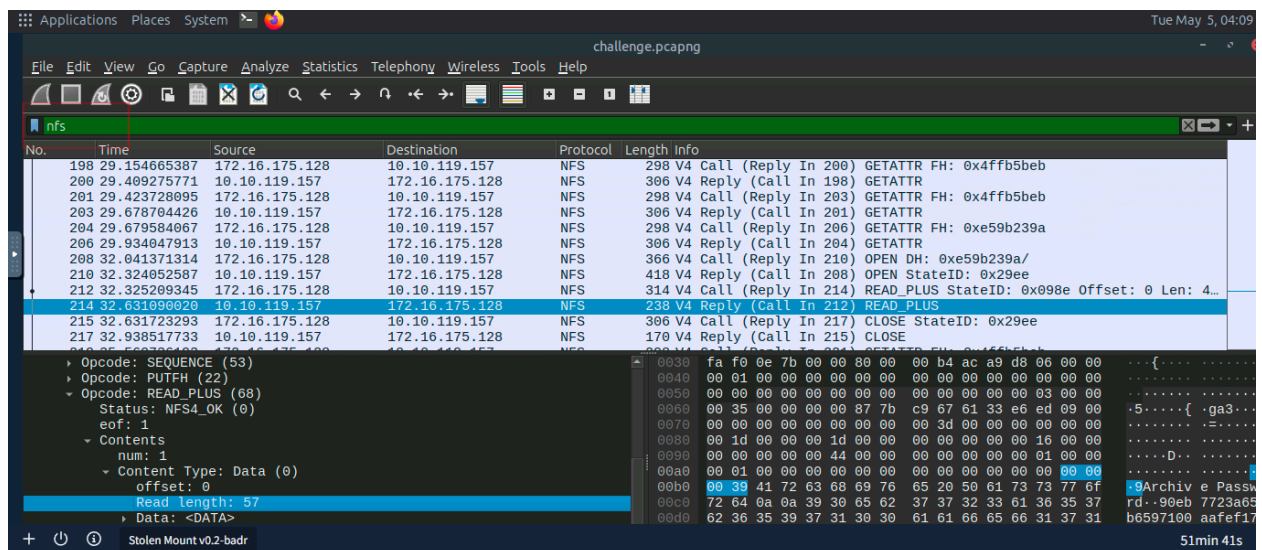
Challenge Explanation

In this scenario, an intruder infiltrated a corporate network and targeted an internal Network File System (NFS) server storing backup files. A classified secret was accessed and exfiltrated. The only artifact left behind is a packet capture (PCAP) file recorded during the incident.

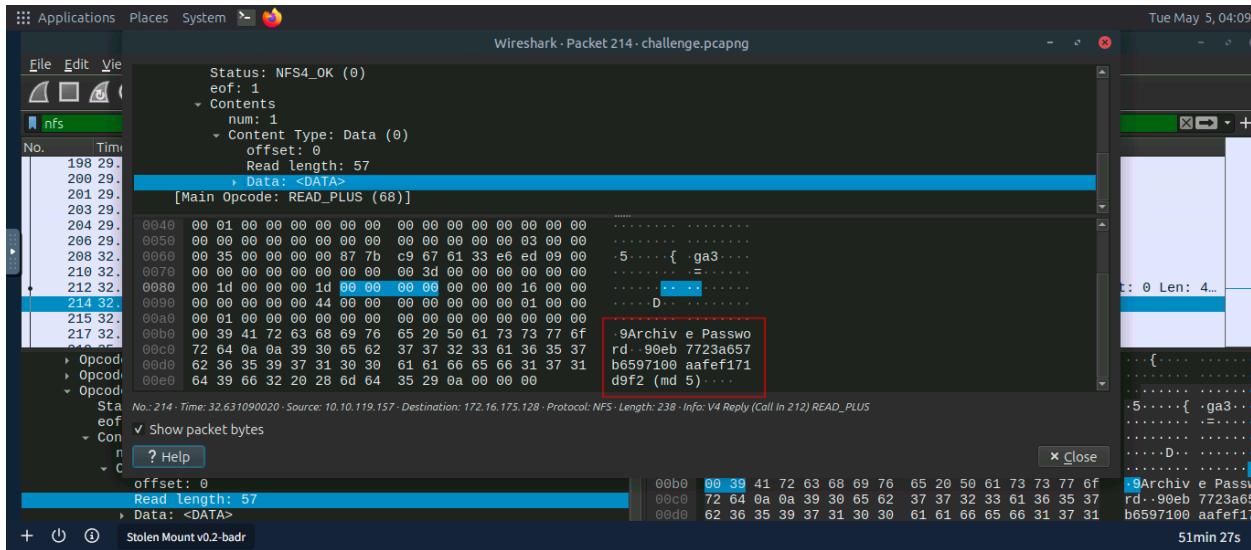
The objective of this challenge is to perform network traffic analysis on the provided PCAP, identify the exfiltrated data, reconstruct the stolen files from the raw packet bytes, and ultimately recover the classified flag.

Challenge Solve

To understand what the attacker was doing, I opened the PCAP in Wireshark and started by filtering for standard NFS traffic using the basic `nfs` display filter. I noticed a series of read requests and replies between the attacker's IP and the NFS server.



While inspecting the **READ_PLUS** replies from the server, I checked the raw packet bytes and found plaintext data sitting in one of the payload sections. It revealed an MD5 hash labeled as the archive password:



Archive Password: 90eb7723a657b6597100aafef171d9f2 (md5)

Knowing that the attacker stole an encrypted archive, I needed to decrypt the hash to obtain the plaintext password.

Enter up to 20 non-salted hashes, one per line:

90eb7723a657b6597100aafef171d9f2

I'm not a robot

This site is exceeding the reCAPTCHA Enterprise free quota.

reCAPTCHA

Crack Hashes

Supports: LM, NTLM, md2, md4, md5, md5(md5_hex), md5-half, sha1, sha224, sha256, sha384, sha512, ripeMD160, whirlpool, MySQL 4.1+ (sha1 sha1_bin)), QubesV3.1BackupDefaults

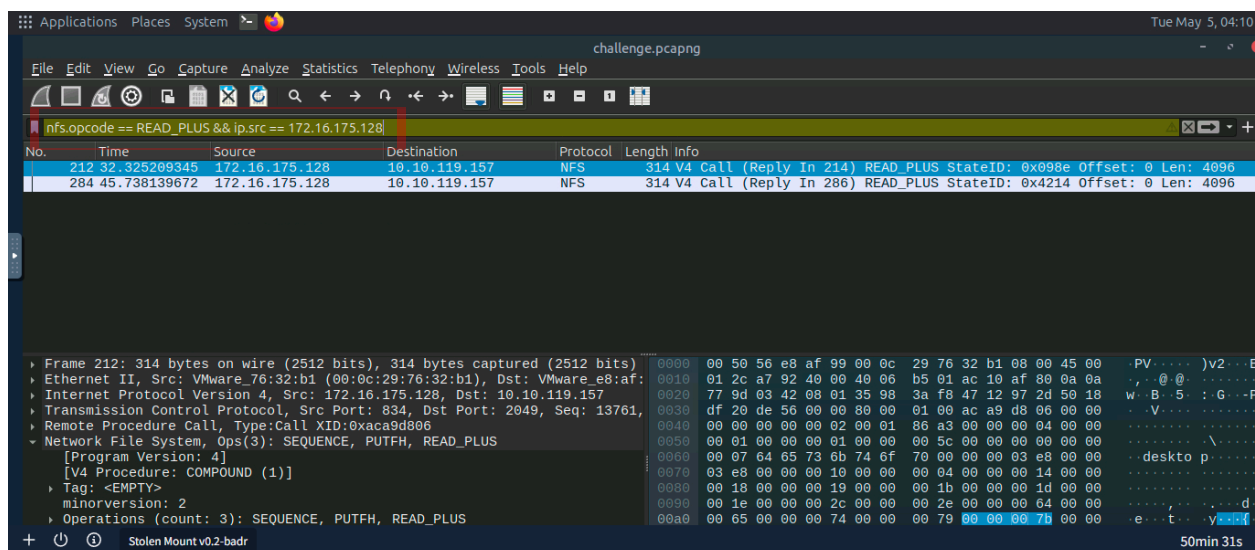
Hash	Type	Result
90eb7723a657b6597100aafef171d9f2	md5	avengers

Upon decrypting the MD5 hash with online decryption tools, I recovered the following plaintext password: *avengers*

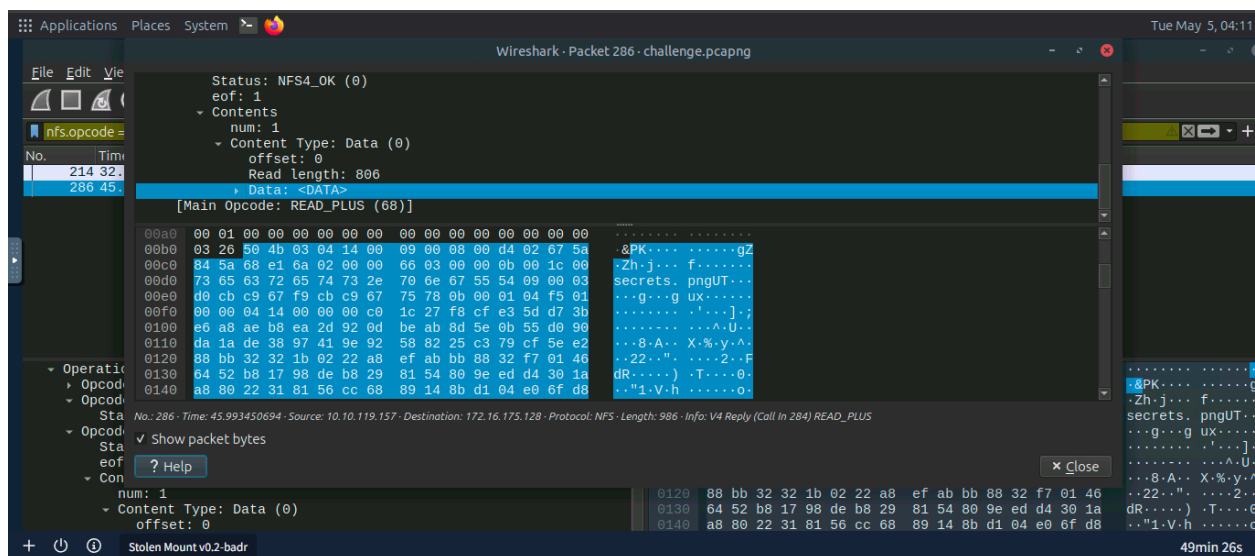
With the password in hand, I needed to locate the actual archive file. To do this, I refined my Wireshark filter to only show the **READ** replies containing data being sent back to the attacker.

(Note: In Wireshark, filtering for the exact operation code is more reliable than using text strings, so I filtered for the specific **READ** opcode being sent to the attacker's destination IP)

`nfs.opcode == READ_PLUS && ip.dst == 172.16.175.128.`



This filter stripped away the noise and left only two major packets. One contained the password text file I had already found, and the other was a large chunk of data containing the file signature **50 4b 03 04** in the hex dump. This is the magic number (file header) for a ZIP archive!

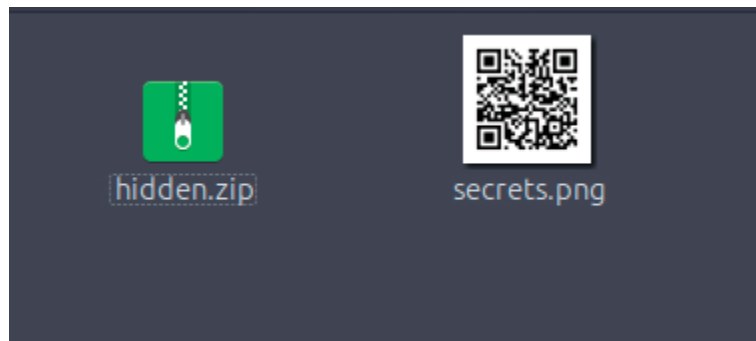


To reconstruct the stolen file, I selected the packet containing the ZIP header. Inside the **Packet Details** pane, I navigated down the NFS protocol tree to the **data** field.

By right-clicking this field and selecting **Export Packet Bytes...**, I was able to save the raw payload directly to my machine and named it `hidden.zip`.

Finally, I unzipped the extracted archive using the cracked password (*avengers*)

```
unzip hidden.zip
# Prompted for password: entered 'avengers'
```



The extraction yielded an image file containing a QR code. After scanning the QR code, the hidden secret was revealed, granting the final flag!

Flag: `THM{n0t_s3cur3_f113_sh4r1ng}`